

CloudFormation: IaC nativo en AWS

▼ CloudFormation: IaC nativo en AWS

Introducción

▼ Templates en CloudFormation

▼ Anatomía de una plantilla de CloudFormation (además de *Resources*)

Ejemplo de estructura mínima de una plantilla

▼ Parámetros (**Parameters**)

Ejemplo de definición y uso de parámetros

▼ Recursos (**Resources**)

Ejemplo de definición de un recurso AWS

▼ Pseudoparámetros

Ejemplo de Uso de pseudoparámetros

▼ Funciones intrínsecas

Ejemplo de Formateo de cadenas con **Fn::Sub**

Ejemplo de Obtener atributos con **Fn::GetAtt**

▼ Outputs (salidas)

Ejemplo de Outputs y exportación de valores

▼ Mappings (Asignaciones)

Ejemplo de Mapping de AMIs por región

▼ Scripts auxiliares para instalación de software

Ejemplo de Uso de CloudFormation-init en una instancia

▼ Condicionales (Conditions)

Ejemplo de Uso de Conditions en recursos

▼ Dependencias entre recursos

Ejemplo de Uso de DependsOn

▼ Validación de plantillas

Ejemplo de Restricciones en parámetros

Buena Prácticas en Diseño de plantillas

Ejemplo de Modularidad con varias pilas

Introducción

AWS CloudFormation es el servicio nativo para IaC de AWS. Permite definir un conjunto de recursos de AWS mediante un archivo de plantilla (YAML o JSON) y desplegarla de manera **orquestada**.

Vistas anteriormente las características fundamentales, ventajas e inconvenientes respecto de otras opciones en IaC, vamos a pasar a tratar todas las características y funcionamiento.

CloudFormation se encarga de crear, actualizar y eliminar los recursos en el orden correcto, manejando automáticamente las **dependencias** entre ellos.



Importante

Al implementar una plantilla, CloudFormation crea una **pila** o *stack* que agrupa todos los recursos.

El servicio **administra el proceso completo**: desde la creación inicial hasta las actualizaciones, aplicando cambios incrementalmente y haciendo cambios si ocurre algún error.

Como vimos, también tiene detección de cambios en la infraestructura fuera del IaC, lo que se denomina **drift detection**, así como **change sets** para previsualizar cambios a implementar en los despliegues y realiza *rollback automático* si falla.

Templates en CloudFormation

Comenzamos con las **plantillas de CloudFormation**, el corazón de la Infraestructura como Código en AWS. Una *plantilla* es un documento (en JSON o YAML) que describe un conjunto de recursos de AWS y sus configuraciones. Al desplegar una plantilla, CloudFormation crea una **pila** (*stack*) con todos los recursos especificados.

Las plantillas suelen incluir al inicio una versión de formato (*AWSTemplateFormatVersion*) y una **Descripción** opcional que explica para qué se va usar, componentes...

```
AWSTemplateFormatVersion: "2010-09-09"  
Description: "Plantilla mínima de ejemplo"  
...
```

A partir de ahí, se estructura en secciones, cada una con un fin específico. La sección **Resources** es la única **obligatoria** y constituye el núcleo: en ella definimos los recursos de la pila (ej. instancias EC2, buckets S3), con un identificador lógico, un tipo y sus propiedades.

El resto de secciones le da flexibilidad, capacidades de orquestación, integración con otros elementos, etc.

Anatomía de una plantilla de CloudFormation (además de *Resources*)

- **Parameters** – Define entradas dinámicas para la plantilla, permitiendo personalizar valores al crear la pila (ej. tipo de instancia, tamaños) bien de otras plantillas o de la interfaz de usuario.
- **Outputs** – Especifica **valores de salida de la pila** (IDs, URLs, etc.) para mostrarlos tras el despliegue o exportarlos a otras pilas. **Los podré usar para encadenar stacks** o simplemente para facilitar la consulta de datos importantes de los recursos creados **desde la consola o CLI**.
- **Mappings** – Tablas estáticas de datos (clave-valor) para usar configuraciones según condiciones, usando por ejemplo función `Fn::FindInMap`.
- **Conditions** – Condicionales lógicos que controlan la creación de recursos o valores según parámetros (p. ej., desplegar cierto recurso solo si `Entorno == "Producción"`).
- **Transform** – Aplicar macros o transformaciones al template (por ejemplo, incluir **plantillas fragmentadas** o usar **AWS::Serverless** para SAM).
- **Metadata** – Con información descriptiva o instrucciones para herramientas) o sección **Rules** (reglas de validación de parámetros) se usan poco.

 [Anatomía de una plantilla de CloudFormation](#)

Ejemplo de estructura mínima de una plantilla

A continuación se muestra una plantilla mínima en **YAML** con sus secciones principales. Incluimos la versión de formato, una descripción, definimos un recurso (un bucket S3) en **Resources**, y un output que devuelve el nombre lógico del bucket:

```
AWSTemplateFormatVersion: "2010-09-09"
Description: "Plantilla mínima de ejemplo"
Resources:
    # Recursos a crear
    MiS3Bucket:
        # Nombre lógico del recurso
        Type: AWS::S3::Bucket
Outputs:
    # Valores de salida de la pila
    NombreDelBucket:
        Value: !Ref MiS3Bucket # Hacemos referencia al recurso
                                # para devolver su nombre como output
```

Parámetros (Parameters)

La sección **Parameters** permite **personalizar** una plantilla pasándole valores en tiempo de despliegue. En otras palabras, podemos definir *parámetros* (tipo, descripción, valores por defecto,

etc.) que el usuario rellenará al crear o actualizar la pila. Gracias a los parámetros, una sola plantilla puede ser **flexible y reutilizable**, adaptándose a distintos entornos sin modificar el código.

Los parámetros se referencian dentro de la plantilla para asignar valores a recursos y salidas. Por ejemplo, se pueden usar con la función intrínseca **Ref** u otras funciones para inyectar el valor proporcionado en propiedades de recursos.

Además, es posible **especificar validaciones básicas**: tipo de dato (String, Number, Lista, etc.), valores permitidos, longitud máxima, patrón regex, etc., para garantizar que el usuario introduzca datos válidos.

En definitiva, **Parameters** actúa como la interfaz de nuestra plantilla, exponiendo opciones configurables (tamaños de instancia, nombres, cantidades) de forma controlada, lo que mejora la **reutilización y modularidad** de las plantillas.

Ejemplo de definición y uso de parámetros

Imaginemos que queremos que nuestra plantilla permita elegir el tipo de instancia EC2. Definimos un parámetro y luego lo usamos en un recurso:

```
Parameters:
2
  InstanceTypeParam:
    Description: "Tipo de instancia EC2"
    Type: String
    Default: t3.small

Resources:
  MiInstanciaEC2:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: ami-0123456789abcdef0
13
      InstanceType: !Ref InstanceTypeParam
      KeyName: testkey
```

Aquí **InstanceTypeParam** es un parámetro de tipo *String* con valor por defecto “t3.small”. En la sección Resources, el recurso **MiInstanciaEC2** usa `!Ref InstanceTypeParam` para asignar el tipo de instancia según el valor que proporcione el usuario (o el default): así se pueden lanzar distintos tamaños de instancia variando sólo el parámetro.

Recursos (Resources)

La sección **Resources** es el núcleo de la plantilla: aquí declaramos cada **recurso AWS** que queremos crear. Cada recurso tiene un **ID lógico** (nombre interno en la plantilla), un **Type** que corresponde a un tipo de recurso de AWS (por ejemplo `AWS::EC2::Instance` para una instancia EC2, `AWS::S3::Bucket` para un bucket S3, etc.) y un bloque de **Properties** donde especificamos la configuración de ese recurso.

CloudFormation soporta cientos de tipos de recursos de prácticamente todos los servicios AWS (EC2, S3, RDS, IAM, Lambda, ECS, etc.). Al crear la pila, CloudFormation traducirá cada definición en recursos reales en AWS. El orden de creado suele ser un problema, por lo que **cloudFormation determina automáticamente las dependencias** entre recursos: si un recurso hace referencia a otro, esperará a que el referido se cree primero. También podemos forzar dependencias manualmente (veremos **DependsOn** más adelante).

En resumen, **cada entrada en Resources equivale a un recurso en la infraestructura**. Es como la lista de “ingredientes” que CloudFormation usará para construir el stack. Una plantilla puede tener desde uno hasta cientos de recursos, todos definidos declarativamente en esta sección.

Ejemplo de definición de un recurso AWS

A continuación, un ejemplo simple de un recurso EC2 definido en la sección **Resources**:

```
Resources:
  MiServidorWeb:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: ami-0abcdef1234567890
      InstanceType: t3.micro
      Tags:
        - Key: Name
          Value: "ServidorWeb"
```

Aquí el recurso lógico **MiServidorWeb** de tipo `AWS::EC2::Instance` representa una instancia EC2. En **Properties** establecemos la AMI (`ImageId`), el tipo de instancia (`InstanceType`) y una etiqueta (Tag) con nombre “ServidorWeb”. CloudFormation se encargará de lanzar esta instancia con esos parámetros.

Podemos agregar tantas propiedades como permita el tipo de recurso (grupo de seguridad, user data, etc.). Si este recurso dependiera de otro (por ejemplo, un VPC o subred), con solo referenciarlos CloudFormation asegura el orden correcto de creación.

Pseudoparámetros

Los **pseudoparámetros** son parámetros predefinidos por CloudFormation que **no necesitan ser declarados** en la plantilla. Están siempre disponibles y proporcionan información del entorno de despliegue o de la propia pila. Se usan igual que un parámetro normal con `**Ref` u otras funciones.

Ejemplos de pseudoparámetros útiles (hay más):

- **AWS::Region** – La región AWS donde se está desplegando la pila.
- **AWS::AccountId** – El ID de cuenta AWS en la que se crea la pila.
- **AWS::StackName** – El nombre de la pila actual.
- **AWS::NoValue** – Indica a CloudFormation que *ignore* o elimine una propiedad (usado en condiciones).

Estos valores ayudan a hacer plantillas más **genéricas**. Por ejemplo, usando `AWS::Region` no hace falta codificar una región específica en la plantilla, sino que toma automáticamente la región actual de despliegue. Lo mismo con `AccountId`, útil para construir ARN dinámicamente, etc.

En resumen, son variables de entorno proporcionadas por CloudFormation para usar dentro de cualquier plantilla.

Ejemplo de Uso de pseudoparámetros

Supongamos que queremos obtener la región AWS en la que estamos desplegando la infraestructura y exponerla como output. Podemos hacerlo sin pedir ningún parámetro al usuario, utilizando el pseudoparámetro **AWS::Region**:

Outputs:

```
RegionDeLaPila:
  # El valor de este output será la región actual
  # de despliegue, gracias al pseudoparámetro AWS::Region
  Value: !Ref "AWS::Region"
```

En este ejemplo, definimos una salida **RegionDeLaPila** cuyo valor es `!Ref AWS::Region`.

CloudFormation sustituirá eso por la región actual (p. ej. “eu-west-1”) cuando creemos la pila. El usuario no pasó nada como parámetro, la plantilla automáticamente capturó el dato del entorno de despliegue.

Del mismo modo podríamos usar `AWS::AccountId` para, por ejemplo, construir el nombre de un bucket único por cuenta, o `AWS::StackName` para incluir el nombre de la pila en alguna configuración.

Los pseudoparámetros **proporcionan contexto del entorno muy útil** dentro de las plantillas sin esfuerzo adicional.

Funciones intrínsecas

CloudFormation ofrece una serie de **funciones intrínsecas** (built-in) para calcular valores dinámicamente dentro de la plantilla. Estas funciones nos permiten, por ejemplo, referenciar recursos y parámetros, hacer cálculos simples, concatenar strings, condicionar la creación de recursos, todo ello durante el *runtime*. Se usan con la sintaxis `Fn::<Nombre>` (en YAML existen abreviaturas con `!`). Algunas de las más comunes son:

- **Ref / Fn::GetAtt** – Proviene de otras partes de la plantilla: `Ref` es el valor de un parámetro o el ID lógico de un recurso, `Fn::GetAtt` obtiene un **atributo** de un recurso (p. ej. URL de bucket S3, ARN de Lambda...)
- **Fn::Join / Fn::Sub** – Manipulan cadenas de texto. `Fn::Join` concatena una lista de subcadenas con un separador dado, mientras que `Fn::Sub` permite insertar valores (parámetros, atributos, pseudoparámetros) dentro de una cadena usando placeholders `${}` de forma muy cómoda.
- **Funciones condicionales** – Como **Fn::If**, **Fn::Equals**, **Fn::And**, **Fn::Or**, **Fn::Not**. Sirven para introducir lógica condicional en la plantilla, por ejemplo crear o no un recurso dependiendo de un valor, o asignar distintos valores a propiedades según una condición booleana.
- **Fn::FindInMap** – Busca un valor dentro de las **Mappings** según claves dadas (ej. dado un mapping de AMLs por región, recuperar la AML correspondiente a la región actual).
- **Fn::ImportValue** – Importa el valor de salida (**Output**) exportado desde otra pila (para recursos entre stacks).

Ejemplo de Formateo de cadenas con Fn::Sub

Veamos un ejemplo de cómo construir dinámicamente un nombre usando `Fn::Sub`. Supongamos que queremos nombrar un bucket S3 incorporando el ID de cuenta AWS para hacerlo único. Podemos combinar un parámetro con un pseudoparámetro:

Parameters:

BucketPrefix:

Type: String

Default: "midemo"

Resources:

MiBucket:

Type: AWS::S3::Bucket

Properties:

Usamos Fn::Sub para construir el nombre del bucket a partir del prefijo y el ID de cuenta

BucketName: !Sub '\${BucketPrefix}-\${AWS::AccountId}'

Aquí usamos **!Sub** para crear el nombre de bucket a partir del prefijo proporcionado (BucketPrefix) y el ID de cuenta AWS actual. Si *BucketPrefix* es "midemo" y la cuenta es 123456789012, CloudFormation asignará el nombre "midemo-123456789012" al bucket. Con `Fn::Sub` insertamos fácilmente variables dentro de strings. Este enfoque evita tener que escribir manualmente el ID de cuenta u otras partes variables, haciendo la plantilla más reutilizable.

Ejemplo de Obtener atributos con **Fn::GetAtt**

Otra situación común es necesitar atributos que salen de un recurso. Por ejemplo, si creamos un bucket S3 para sitio web estático, CloudFormation puede darnos la URL del sitio web. Usamos **Fn::GetAtt** indicando el recurso y el atributo deseado:

Resources:

MiBucket:

Type: AWS::S3::Bucket

Properties:

Al definir el bucket como sitio web, CloudFormation

nos dará un atributo WebsiteURL con la URL pública del sitio

WebsiteConfiguration:

IndexDocument: index.html

Outputs:

URLSitioWeb:

Mostramos como output la URL del sitio web

obtenida del atributo WebsiteURL del bucket

Value: !GetAtt MiBucket.WebsiteURL

En este caso, definimos **MiBucket** con la configuración de sitio estático (estableciendo *IndexDocument*). Luego, en Outputs, obtenemos la URL del sitio web (*websiteURL*) mediante

`!GetAtt` . Cuando la pila se cree, CloudFormation rellena ese output con la URL pública del bucket para web (por ejemplo, `MiBucket.s3-website.eu-west-1.amazonaws.com`).

En resumen es útil para obtener detalles de recursos y usarlos en *outputs* u otros recursos.

Outputs (salidas)

La sección **Outputs** permite exponer información útil de la pila una vez creada. Podemos pensar en los outputs como “**resultados**” que CloudFormation nos devuelve: por ejemplo, la URL pública de un sitio web, el endpoint de una base de datos, el ID de un VPC creado, etc. Estas salidas aparecen en la consola de CloudFormation al finalizar el despliegue y también se pueden obtener por CLI/API.

Los outputs sirven para **encadenar** stacks o simplemente para facilitar operaciones manuales. Se pueden marcar outputs para **exportación**, asignándoles un nombre global. Esto permite que otras pilas las **importen** (usando `Fn::ImportValue`), facilitando el intercambio de información entre stacks.

En resumen, **los outputs dan visibilidad de los recursos creados**. En una plantilla bien diseñada, deberíamos exponer los datos que los administradores o otras plantillas necesitarán, en lugar de obligar a buscarlos en la consola.

Por ejemplo, en una plantilla que crea un ALB, podríamos tener un output con la URL DNS del ALB, o en una que crea un RDS, un output con la cadena de conexión, etc. Esto hace la vida más fácil tras el despliegue y **habilita la composición de arquitecturas usando múltiples pilas conectadas**.

Ejemplo de Outputs y exportación de valores

Supongamos que nuestra plantilla crea un VPC y queremos que su ID esté disponible para otras pilas. Podemos definir una salida y **exportarla**:

```
Resources:
  MiVPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: 10.0.0.0/16

Outputs:
  VPCId:
    Value: !Ref MiVPC
    Export:
      Name: MiVPC-ID
```

Aquí el output **VPCId** toma el ID del VPC creado (`!Ref MiVPC`) y lo exporta con el nombre “MiVPC-ID”. Otro stack podría usar `Fn::ImportValue "MiVPC-ID"` para obtener este ID y reutilizar la red. Por ejemplo, una plantilla de *Aplicación* podría importar la VPC y subredes definidas por una plantilla de *Red*. De esta manera, Outputs/Exports permiten *composición de stacks*: dividir la infraestructura en módulos pero conectarlos a través de valores compartidos. En la consola, tras crear la pila, veremos el ID de la VPC en la sección **Outputs**, listo para usarse o consultarse fácilmente.

Mappings (Asignaciones)

Las **Mappings** son una sección donde **definimos datos estáticos en forma de tabla de búsqueda**, útiles para evitar lógica compleja dentro de la plantilla. Funcionan como un diccionario anidado: clave principal, clave secundaria y valor. Por ejemplo, podemos mapear regiones AWS a ciertos valores específicos de cada región (AMI IDs, AZs, etc.), o tipos de entorno a configuraciones distintas.

Dentro de la plantilla, obtenemos valores de un mapping con la función

`Fn::FindInMap("Mapa", "Clave1", "Clave2")` . Esto nos devuelve el valor correspondiente a esa combinación de claves.

Un uso típico es almacenar IDs de AMI por región: definimos un mapping donde la clave principal es la región (`us-east-1` , `eu-west-1` , etc.) y la secundaria podría ser “AMI”, asignando el ID apropiado para cada región. Luego con `FindInMap` recuperamos el AMI según la región actual (`AWS::Region`). Así, en lugar de tener una sola AMI rígida, la plantilla elige la correcta según dónde se despliegue.

En resumen, **Mappings** supone una forma sencilla de manejar valores condicionales o dependientes de contexto sin usar lógica ni parámetros – son tablas de constantes que la plantilla puede consultar rápidamente. Son ideales cuando las combinaciones posibles son conocidas y acotadas (p.ej., AMI por región, tamaños por entorno, etc.).

Ejemplo de Mapping de AMIs por región

```

Mappings:
2
    RegionMap:
        us-east-1:
5
6
            AMI: ami-0aaabbbccc1112222
        eu-west-1:
            AMI: ami-0xxxyyyzzz8887777

Resources:
    Servidor:
        Type: AWS::EC2::Instance
        Properties:
15
            ImageId: !FindInMap [RegionMap, !Ref "AWS::Region", AMI]
            InstanceType: t3.micro
```

En este ejemplo, **RegionMap** es un mapping con entradas para `us-east-1` y `eu-west-1`, cada una proporcionando una AMI. El recurso **Servidor** usa `!FindInMap [RegionMap, AWS::Region, AMI]`. CloudFormation buscará la clave de región actual (ej. `eu-west-1`) dentro de `RegionMap` y extraerá el valor `AMI`, resultando en el ID correcto para esa región.

Así, la misma template puede lanzar instancias EC2 en distintas regiones usando la AMI apropiada automáticamente, sin necesitar parámetros ni condicionales complicados, al estilo de las constantes clásicas en programación.

Scripts auxiliares para instalación de software

CloudFormation no solo crea infraestructura “en bruto”: también facilita la configuración inicial de servidores mediante **helper scripts**. AWS proporciona scripts como **CloudFormation-init**, **CloudFormation-signal**, **CloudFormation-hup**, etc., que se pueden usar dentro de instancias EC2 para automatizar la instalación de software, configuración del sistema y notificación de estado.

Por ejemplo, con **CloudFormation-init** podemos definir en la plantilla, dentro de la sección **Metadata**, una serie de tareas: **paquetes a instalar**, **archivos de configuración a crear**, **servicios a iniciar**, **comandos a ejecutar**, etc. Luego, en el *User Data* de la instancia (el script de arranque), llamamos a *CloudFormation-init* para que lea esas instrucciones y las ejecute durante el boot. De esta forma, una

instancia puede arrancar y convertirse en un servidor web con Apache instalado y configurado, orquestado por CloudFormation.

Asimismo, **CloudFormation-signal** se usa para notificar a CloudFormation cuando una instancia ha terminado su configuración (por ejemplo, para pasar a espera o Auto Scaling Rolling Updates).

CloudFormation-hup puede detectar cambios en Metadata dentro de la instancia y aplicar updates...

En resumen, estos scripts auxiliares permiten tratar la configuración de software como parte del despliegue IaC, logrando que una plantilla CloudFormation además de crear una instancia la deje lista con el software (**bootstrapping**).  [Más detalle en estos ejemplos](#)

Ejemplo de Uso de CloudFormation-init en una instancia

Supongamos que queremos preparar una instancia EC2 con una aplicación web. Podemos usar en Metadata **AWS::CloudFormation::Init** para describir los pasos, y luego CloudFormation-init para ejecutarlos. Por ejemplo:

Resources:

Servidor:

Type: AWS::EC2::Instance

Sección de Metadata con instrucciones para CloudFormation-init

Metadata:

Al iniciar la instancia, CloudFormation-init ejecutará estas tareas:

AWS::CloudFormation::Init:

config:

packages:

Instalar paquetes con el gestor de paquetes yum

yum:

httpd: [] # Instalar Apache

files:

/var/www/html/index.html:

content: "<h1>¡Hola desde CloudFormation!</h1>"

services:

sysvinit:

httpd:

enabled: true

ensureRunning: true

Properties:

ImageId: ami-0123456789abcdef0

InstanceType: t2.micro

UserData:

Fn::Base64: !Sub |

#!/bin/bash

/opt/aws/bin/CloudFormation-init -v --stack \${AWS::StackName} --resource Servidor --region

/opt/aws/bin/CloudFormation-signal -e \$? --stack \${AWS::StackName} --resource Servidor

En este ejemplo, en **Metadata** definimos que yum instale *httpd*, que se cree un archivo index.html sencillo, y que se habilite/inicie el servicio httpd. Luego, en **UserData**, invocamos

CloudFormation-init para que configure todo según lo definido y **CloudFormation-signal** para notificar si tuvo éxito, usando el código de salida `$?`).

El resultado: CloudFormation lanzará la instancia, CloudFormation-init dentro de ella instalará Apache, colocará la página web y levantará el servicio, y CloudFormation-signal avisará a CloudFormation de que el servidor está listo. Todo esto ocurre automáticamente al desplegar la pila, sin intervención manual.

Es una opción potente, aunque compleja según el proceso concreto, para instalaciones de software en las máquinas aprovisionadas.

Condicionales (Conditions)

La sección **Conditions** nos permite definir **condiciones lógicas** que luego podemos aplicar para controlar la creación de recursos o la asignación de valores en base a algún parámetro o entorno.

Podemos crear una condición, por ejemplo *EsEntornoProd*, que evalúe si el parámetro *Entorno* es “prod”. Luego, en un recurso, podemos añadir la propiedad **Condition: EsEntornoProd** para que ese recurso **sólo se cree** si la condición es verdadera. Igualmente, en las propiedades se puede usar la función `Fn::If` para elegir un valor u otro dependiendo de la condición.

Las condiciones se evalúan al inicio del despliegue de la pila, en función de los parámetros u otros valores disponibles. **Gracias a ellas, una plantilla puede abarcar múltiples escenarios: por ejemplo, una misma plantilla podría servir tanto para entornos de desarrollo (creando recursos más pequeños o menos recursos) como para producción (más recursos, con alta disponibilidad)**, encendiendo/apagando partes según lo indicado. Esto evita tener plantillas separadas casi iguales, con *Conditions* incluimos la variabilidad de forma limpia.

Internamente, las Conditions usan las funciones intrínsecas condicionales (`Fn::If` , `Fn::Equals` , etc.) para sus expresiones, dando **lógica** dentro del entorno declarativo de CloudFormation, de manera controlada.

Ejemplo de Uso de Conditions en recursos

Parameters:

Entorno:

Type: String

Default: dev

AllowedValues: [dev, prod]

7

Conditions:

EsProd: !Equals [!Ref Entorno, "prod"]

Resources:

BackupVolume:

Type: AWS::EC2::Volume

14

15

Condition: EsProd

Properties:

Size: 100

VolumeType: gp3

AvailabilityZone: !Select [0, !GetAZs ""]

Con la condición **EsProd** se crea un volumen EBS adicional para backups, pero en desarrollo no. Observa el uso de `!Select` y `!GetAZs` para elegir la primera AZ disponible – así el volumen se crea en la zona 0 por simplicidad en este ejemplo. Este patrón es muy útil para toggle de recursos enteros (p.ej., activar un Load Balancer solo si se indica alta disponibilidad, etc).

Dependencias entre recursos

CloudFormation normalmente infiere el orden de creación a partir de las referencias: si un recurso A referencia a B (por ejemplo con `Ref`), sabe que B debe crearse antes que A. Esto se llama **dependencia implícita**. Sin embargo, en algunas situaciones puede que no haya una referencia directa pero aún así necesitamos forzar un orden. Para ello existe el atributo **DependsOn** en los recursos, donde podemos listar uno o varios recursos de los que debe depender.

Por ejemplo, ciertas configuraciones de red requieren secuencia explícita: un *Internet Gateway* debe existir antes de asociarlo a un VPC. Si la asociación no referencia directamente el IGW (sino solo IDs), podríamos usar **DependsOn** para asegurarnos. Asimismo, CloudFormation realiza muchas operaciones en paralelo para acelerar el despliegue: con **DependsOn** podemos indicar “no hagas esto en paralelo, espera a X primero” en casos necesarios.

Casos comunes de uso: recursos que deben ser eliminados en orden específico (DependsOn garantiza que en la eliminación, hará el inverso), o inicializaciones donde un recurso necesita que otro esté 100% operativo primero.

En general, es buena práctica confiar en las dependencias implícitas siempre que sea posible, pero **DependsOn** puede ser necesario en ciertos casos. Nos da control manual del grafo de creación cuando la inferencia automática no basta o no existe.

Ejemplo de Uso de DependsOn

```
Resources:
```

```
  MyVPC:
```

```
    Type: AWS::EC2::VPC
```

```
    Properties:
```

```
      CidrBlock: 10.0.0.0/16
```

```
7
```

```
    Type: AWS::EC2::InternetGateway
```

```
  AttachGateway:
```

```
    Type: AWS::EC2::VPCGatewayAttachment
```

```
    Properties:
```

```
      VpcId: !Ref MyVPC
```

```
      InternetGatewayId: !Ref MyInternetGateway
```

```
15
```

```
17
```

```
    DependsOn: MyInternetGateway
```

Aquí el recurso **AttachGateway** depende explícitamente de **MyInternetGateway**. Aunque en *Properties* ya referenciamos ambos IDs (lo que normalmente crearía dependencias implícitas), este ejemplo muestra la sintaxis. Algunos recursos de attachment en networking *sí requieren* DependsOn debido a timings internos (en ciertos casos CloudFormation intenta el attach demasiado rápido). Este patrón garantizaría un orden de orquestación cuando la simple referencia no es suficiente o no existe, con múltiples dependencias.

Validación de plantillas

Antes de desplegar una plantilla, CloudFormation proporciona [una validación básica](#) de **sintaxis**: por ejemplo, mediante la AWS CLI (`aws cloudformation validate-template`) o al cargarla en la consola, se comprueba que el JSON/YAML está bien formado y que las secciones y sintaxis son correctas.

Sin embargo, esta validación **no garantiza** que los valores de propiedades sean válidos ni que los recursos se creen (no detecta, por ejemplo, si pediste 500 instancias y tu cuota es menor). Para validaciones más profundas existen herramientas como **CloudFormation-lint**, que verifica la plantilla contra el esquema de recursos AWS y mejores prácticas.

Dentro de la plantilla, podemos definir restricciones en los **Parameters** para asegurar cierta validez: *AllowedValues*, *AllowedPattern* (regex para el formato), *Min/MaxLength*, *Min/MaxValue*, *ConstraintDescription* (mensaje de error)... para evitar que el usuario que lanza la plantilla introduzca parámetros fuera de rango.

Para casos más complejos, CloudFormation introdujo la sección **Rules**, con parámetros condicionados unos sobre otros por ejemplo, que tienen un uso menos habitual.

Ejemplo de Restricciones en parámetros

```
Parameters:
  AppName:
    Description: "Nombre de la aplicación"
    Type: String
    AllowedPattern: "^[a-z0-9]{3,8}$"
    ConstraintDescription: "debe tener de 3 a 8 caracteres alfanuméricos en minúscula"
```

En este ejemplo, **AllowedPattern** define una expresión regular que el valor debe cumplir (entre 3 y 8 caracteres sólo letras minúsculas y números). Si el usuario introduce algo que no coincide (por ejemplo “App1” con mayúscula A o demasiado corto), CloudFormation no iniciará la creación de la pila y en la consola mostrará el mensaje de *ConstraintDescription* (“debe tener de 3 a 8 caracteres alfanuméricos en minúscula”).

De modo similar, podríamos usar **AllowedValues** para una lista de valores concretos. Por ejemplo:

```
AllowedValues: ["small", "medium", "large"]
```

Esto obligaría a que el valor sea exactamente uno de esos tres. Estas restricciones hacen la plantilla más robusta, guiando al usuario y previniendo errores comunes, ya que se validan antes de crear el recurso.

Buena Prácticas en Diseño de plantillas

Más allá de la sintaxis, se suelen recomendar algunas **buenas prácticas**:

- **Documentación:** Aprovecha el campo **Description** de la plantilla para describir su propósito. Usa **nombres lógicos claros para recursos y parámetros**. Puedes incluir **Metadata** adicional o comentarios YAML para explicar secciones complejas. Esto ayuda a otros (y a tu yo futuro) a entender la template.
- **Modularidad:** Divide infraestructuras grandes en plantillas más pequeñas por dominios o capas (red, base de datos, frontend, etc.). Puedes utilizar **nested stacks** (luego los veremos) o *exports/Imports* para conectarlas, evitando una mega plantilla monolítica difícil de mantener.
- **Reutilización:** Escribe plantillas genéricas usando parámetros en lugar de valores fijos. Emplea *Mappings* para casos especiales por región o entorno. La idea es no duplicar lógica en múltiples templates, sino una plantilla adaptable. Si te ves copiando y pegando recursos entre plantillas, piensa en factorizar esa parte común.
- **Evitar valores sensibles en texto plano:** No incluyas secretos (passwords, claves) en las templates. Usa *Parameters* de tipos integrados como *SecureString*, **AWS Secrets Manager** o *SSM Parameter Store* y referencias a ellos, para obtener credenciales de forma segura.
- **Pruebas y validación continua:** Versiona tus plantillas (Git) y pruébalas en entornos de prueba antes de producción. Utiliza herramientas como **CloudFormation-lint** (linting estático) y **taskcat** (herramienta de testeo y despliegue de plantillas en múltiples regiones) para detectar problemas pronto. Integrar estas pruebas en tu pipeline CI/CD te ahorrará sorpresas.
- **Evitar drift:** Siempre que sea posible, realiza cambios de infraestructura a través de CloudFormation (ya sea actualizando la pila o usando Change Sets). No manualmente en la consola. Esto **mantiene la plantilla como fuente de verdad** y permite usar detección de desviaciones para verificar integridad.

Si hay recursos que no quieres que CloudFormation toque (por ejemplo datos críticos), considera políticas de eliminación (*DeletionPolicy*) en *Retain* o *Snapshot*, en lugar de cambios manuales fuera de CloudFormation.

- **Límites:** Ten en cuenta los límites de CloudFormation (tamaño máximo de plantilla ~51 KB en línea, 200 recursos por stack por defecto, etc.). Si tu plantilla crece demasiado, es señal para refactorizar modularmente.

También fíjate en el tiempo: las operaciones muy largas pueden agotar timeouts, en cuyo caso puedes usar estrategias como crear recursos en lotes o aumentar Timeouts en *wait conditions*.

Ejemplo de Modularidad con varias pilas

Por ejemplo, supongamos tener la infraestructura partida en dos plantillas 'Network.yaml' y 'App.yaml':

```
# plantilla Network.yaml
Resources:
  NetVPC:
    Type: AWS::EC2::VPC
    Properties: { CidrBlock: 10.0.0.0/16 }
Outputs:
  VPCIdExport:
    Value: !Ref NetVPC
    Export:
      Name: MyNetworkStack-VPCID

# plantilla App.yaml (asume que Network se ejecutó primero)
Resources:
  AppServer:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: ami-0123456789abcdef0
      InstanceType: t3.small
      NetworkInterfaces:
        - SubnetId: !ImportValue MyNetworkStack-VPCID
          DeviceIndex: 0
```

En el código anterior, la plantilla de Red **exporta** el ID del VPC y la de Aplicación lo **importa**. En concreto, la plantilla de aplicación importa el VPC Id exportado por la de red usando **!ImportValue**.

De esta forma, podemos desplegar la red una vez y reutilizarla para múltiples stacks de aplicaciones. Cada equipo (red, aplicación, base de datos, etc.) puede tener su plantilla, y todas encajan gracias a estos enlaces explícitos.

Esto mejora la **organización por ciclo de vida y responsabilidad**: por ejemplo, la red puede mantenerse sin tocar las apps y viceversa.

Además, **acelera despliegues (podemos actualizar una parte sin redespargar todo) y evita duplicación**.

La modularidad mediante múltiples pilas bien acopladas es una de las claves para **escalar laC en proyectos grandes**.